UTS

PEMBELAJARAN MESIN



Nama	: Naura Putri Ahda Asty
NIM	: 202431098
Kelas	: G
Nama Dosen	: Dr. Dra. Dwina Kuswardani, M.Kom
No. Bangku	: -
Nama Asisten	:

1 Muhammad Ichwan Fauzi

2. Morga Immanuel Nainggolan

LABORATORIUM COMPUTER NETWORK

INSTITUT TEKNOLOGI PLN

2026

DAFTAR ISI

DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **1**
1.1 Rumusan Masalah 1
1.2 Tujuan Masalah 1
1.3 Manfaat Masalah 2
BAB II LANDASAN TEORI **3**
2.1 Pembelajaran Mesin Terbimbing dan Prediksi Diabetes 3
2.2 Exploratory Data Analysis dan Preprocessing 3
2.3 K-Nearest Neighbors (KNN) 4
2.4 Regresi Linear 4
2.5 Metrik Evaluasi 5
2.6 Validasi Silang dan Pemilihan Hiperparameter 5
2.7 Ketidakseimbangan Kelas dan Penentuan Threshold 6
2.8 Perbandingan Karakteristik KNN dan Regresi Linear 6
BAB III HASIL **7**
3.1 Import Library 7
3.2 Memuat Dataset 7
3.3 Informasi Umum Dataset (EDA) 8
3.4 Pemeriksaan Nilai Kosong dan Duplikat 8
3.5 Statistik Deskriptif 9
3.6 Distribusi Kelas Target 9
3.7 Distribusi Fitur Kategorikal 10
3.8 Distribusi Fitur Numerik 11
3.9 Korelasi Antar Fitur 12
3.10 Preprocessing: Menghapus Data Duplikat 13
3.12 Preprocessing: Pembagian Data dan Standardisasi 14
3.13 KNN Baseline: Dengan dan Tanpa Scaling (K = 5) 14
3.14 Pemilihan Nilai K dengan 5-Fold Cross Validation 15
3.15 Evaluasi Model KNN pada Data Uji 17
3.16 Regresi Linear: Pelatihan dan Metrik Regresi 19
3.17 Distribusi Skor Prediksi Regresi Linear 19

3.18 Klasifikasi Regresi Linear dengan Threshold 20
3.19 Perbandingan Model KNN dan Regresi Linear 21
BAB IV PENUTUP 23
DAFTAR PUSTAKA iii

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

Penyakit diabetes melitus merupakan salah satu penyakit kronis dengan jumlah penderita yang terus meningkat, sehingga deteksi dini sangat penting untuk membantu tenaga kesehatan mengambil keputusan lebih cepat. Pada UTS Pembelajaran Mesin ini diberikan dataset diabetes (02_diabetes.csv) berisi 100.000 catatan pasien dengan delapan atribut prediktor (gender, age, hypertension, heart_disease, smoking_history, bmi, HbA1c_level, blood_glucose_level) dan satu atribut target (diabetes). Berdasarkan pembagian algoritma kelas TI-G, algoritma yang ditugaskan kepada penulis adalah K-Nearest Neighbors (KNN) dan Regresi Linear.

Rumusan masalah dari praktikum UTS ini adalah sebagai berikut:

1. Bagaimana melakukan Exploratory Data Analysis (EDA) dan preprocessing pada dataset diabetes agar data siap dilatih oleh model?
2. Bagaimana penerapan algoritma KNN dan Regresi Linear untuk memprediksi status diabetes pasien pada dataset tersebut?
3. Berapa akurasi yang diperoleh masing-masing algoritma, dan bagaimana perbandingan kinerja keduanya?

1.2 Tujuan Masalah

Tujuan dari pelaksanaan praktikum Pembelajaran Mesin ini adalah untuk memberikan pemahaman yang lebih mendalam kepada mahasiswa mengenai konsep dasar serta penerapan Machine Learning dalam proses pengolahan dan analisis data. Selain itu, tujuan ini juga menjadi acuan dalam menjawab rumusan masalah yang telah dirumuskan sebelumnya. Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan mampu menjelaskan konsep dasar Machine Learning dan Workflow Data serta memahami tahapan-tahapan yang dilakukan dalam proses pengolahan data.
2. Mahasiswa dapat memahami proses Web Scraping dan pengumpulan data secara otomatis serta mengetahui bagaimana data dapat diperoleh untuk digunakan dalam proses analisis dan pengolahan data.
3. Mahasiswa diharapkan mampu memahami proses Exploratory Data Analysis (EDA) dan Data Preprocessing, termasuk pemeriksaan, pembersihan, dan persiapan data sebelum digunakan dalam pemodelan.
4. Mahasiswa dapat memahami konsep pemodelan Regresi Linear serta mengetahui bagaimana Regresi Linear digunakan untuk mengetahui hubungan antarvariabel dan melakukan prediksi berdasarkan data yang tersedia.
5. Mahasiswa diharapkan mampu membedakan Regresi Linear Sederhana dan Regresi Linear Berganda serta memahami penerapan kedua metode tersebut dalam proses pemodelan dan prediksi pada data praktikum.
6. Mahasiswa dapat menerapkan tahapan pengolahan dan analisis data dalam praktikum, mulai dari membaca dataset, membersihkan data, melakukan analisis dan visualisasi, hingga membuat serta mengevaluasi model Machine Learning.

1.3 Manfaat Masalah

Manfaat dari pelaksanaan praktikum Pembelajaran Mesin ini adalah untuk menambah pengetahuan dan pengalaman mahasiswa dalam memahami serta menerapkan konsep Machine Learning pada data. Melalui praktikum ini, mahasiswa tidak hanya memahami materi secara teori, tetapi juga dapat menerapkannya secara langsung dalam proses pengolahan, analisis, dan pemodelan data. Adapun manfaat dari praktikum ini adalah sebagai berikut:

1. Memahami alur kerja pembelajaran mesin secara utuh, mulai dari eksplorasi data, preprocessing, pelatihan, hingga evaluasi model.
2. Memahami perbedaan karakteristik algoritma berbasis jarak (KNN) dan algoritma berbasis persamaan linear (Regresi Linear) pada kasus klasifikasi biner.
3. Memahami bahwa akurasi saja tidak cukup untuk data yang tidak seimbang, sehingga metrik presisi, recall, dan F1-score perlu dipertimbangkan.
4. Memperoleh gambaran awal bagaimana model pembelajaran mesin dapat mendukung skrining risiko diabetes.

BAB II

LANDASAN TEORI

2.1 Pembelajaran Mesin Terbimbing dan Prediksi Diabetes

Pembelajaran mesin terbimbing (supervised learning) adalah pendekatan ketika model dilatih menggunakan data berlabel, yaitu pasangan fitur masukan dan target keluaran, sehingga model dapat memprediksi target pada data baru. Apabila targetnya berupa kategori (misalnya diabetes atau tidak), tugasnya disebut klasifikasi; apabila targetnya berupa nilai kontinu, tugasnya disebut regresi. Dalam bidang kesehatan, pendekatan ini banyak dipakai untuk memprediksi timbulnya diabetes. Wang (2024) mengulas penggunaan algoritma seperti KNN, SVM, dan random forest untuk prediksi diabetes, sedangkan Aliyu et al. (2024) menunjukkan bahwa kualitas data, khususnya keseimbangan kelas, sangat memengaruhi kinerja klasifikasi diabetes.

2.2 Exploratory Data Analysis dan Preprocessing

Exploratory Data Analysis (EDA) adalah tahap memahami data melalui statistik deskriptif dan visualisasi, misalnya memeriksa tipe data, nilai kosong, duplikasi, distribusi tiap atribut, distribusi kelas target, serta korelasi antar atribut. Hasil EDA menentukan langkah preprocessing yang diperlukan. Beberapa langkah preprocessing yang relevan pada praktikum ini adalah:

- **Penghapusan data duplikat**, agar baris yang sama tidak dihitung berulang dan tidak menimbulkan kebocoran data antara data latih dan data uji.
- **Encoding fitur kategorikal.** Algoritma KNN dan Regresi Linear hanya dapat memproses angka, sehingga fitur kategorikal (gender dan smoking_history) diubah menjadi variabel biner dengan one-hot encoding.
- **Pembagian data latih dan uji.** Data dibagi menjadi data latih untuk membangun model dan data uji untuk menilai kemampuan generalisasi; pembagian berstrata menjaga proporsi kelas tetap sama pada kedua bagian.
- **Penskalaan fitur (standardisasi).** Standardisasi mengubah setiap fitur menjadi $z = (x - \mu) / \sigma$ sehingga memiliki rata-rata 0 dan simpangan baku 1. Langkah ini penting bagi algoritma berbasis jarak karena fitur dengan rentang besar (misalnya blood_glucose_level) akan mendominasi perhitungan jarak. Allorerung et al. (2024) membandingkan beberapa metode normalisasi pada klasifikasi KNN untuk dataset penyakit dan menemukan bahwa metode terbaik bergantung pada karakteristik dataset, sedangkan Assegie et al. (2023) menggunakan data yang telah diskalakan (min-max) sebelum melatih KNN untuk prediksi diabetes. Scaler hanya di-fit pada data latih agar informasi data uji tidak bocor ke proses pelatihan.

Selain itu, pencilan (outlier) perlu dikenali melalui boxplot atau statistik deskriptif. Pencilan tidak selalu dibuang: nilai ekstrem yang masih mungkin terjadi secara medis, misalnya indeks massa tubuh (BMI) yang sangat tinggi, dapat membawa informasi penting. Pada one-hot encoding, salah satu kategori dihilangkan (drop_first) untuk

menghindari redundansi antar kolom dummy yang dapat menimbulkan multikolinearitas sempurna pada regresi linear.

2.3 K-Nearest Neighbors (KNN)

KNN adalah algoritma berbasis instance (lazy learner) yang tidak membangun model eksplisit pada tahap pelatihan, melainkan menyimpan seluruh data latih. Untuk memprediksi sebuah data baru, KNN menghitung jarak data tersebut ke seluruh data latih, memilih K tetangga terdekat, lalu menentukan kelas berdasarkan suara terbanyak (majority voting) dari K tetangga tersebut. Ukuran jarak yang umum digunakan adalah jarak Euclidean:

$$d(x, y) = \sqrt{(\Sigma_i (x_i - y_i)^2)}$$

Hameed dan Joshi (2024) menunjukkan bahwa pemilihan jenis jarak sangat memengaruhi akurasi KNN pada prediksi diabetes; pada dataset PIMA, akurasi terbaik sebesar 80,5% diperoleh dengan K = 35 dan jarak Euclidean. Assegie et al. (2023) juga memperlihatkan bahwa penyetelan nilai K melalui grid search meningkatkan akurasi KNN sebesar 5,29% hingga 82,5% pada prediksi diabetes. Nilai K yang terlalu kecil membuat model sensitif terhadap noise (overfitting), sedangkan K yang terlalu besar membuat batas keputusan terlalu halus (underfitting). Karena itu K dipilih secara empiris, misalnya dengan k-fold cross validation, yaitu membagi data latih menjadi k lipatan dan bergantian memakai satu lipatan sebagai data validasi.

Kelebihan KNN adalah konsepnya sederhana, tidak mengasumsikan bentuk hubungan tertentu antara fitur dan target, serta mampu membentuk batas keputusan non-linear. Kekurangannya, waktu prediksi meningkat seiring jumlah data latih karena jarak harus dihitung ke seluruh data, kebutuhan memori besar karena seluruh data latih disimpan, dan kinerjanya menurun jika terdapat banyak fitur yang tidak relevan atau skala fitur tidak seragam. Pada kasus kelas tidak seimbang, tetangga terdekat cenderung didominasi kelas mayoritas sehingga kelas minoritas sulit terdeteksi, yang tercermin pada recall yang rendah.

2.4 Regresi Linear

Regresi linear memodelkan hubungan antara satu atau lebih fitur dengan target kontinu menggunakan persamaan linear:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

Koefisien β diestimasi dengan metode kuadrat terkecil (ordinary least squares) yang meminimalkan jumlah kuadrat selisih antara nilai aktual dan nilai prediksi. Kualitas model regresi diukur dengan MAE, MSE, RMSE, dan koefisien determinasi R^2, yaitu proporsi variasi target yang dapat dijelaskan model. Hanafi et al. (2024) menerapkan regresi linear untuk memprediksi kadar gula darah pasien diabetes dan melaporkan bahwa regresi linear mengungguli Lasso, Ridge, XGBoost, dan KNN pada penelitian tersebut.

Regresi linear bekerja baik apabila beberapa asumsi terpenuhi, yaitu hubungan antara fitur dan target bersifat linear, galat saling bebas, ragam galat konstan (homoskedastisitas), galat berdistribusi mendekati normal, dan tidak terdapat

multikolinearitas tinggi antar fitur. Karena itu matriks korelasi antar fitur diperiksa pada tahap EDA. Koefisien yang diperoleh pada fitur yang telah distandardisasi juga dapat dibandingkan langsung untuk menilai fitur mana yang paling berpengaruh terhadap prediksi, sebab semuanya berada pada skala yang sama.

Pada praktikum ini target diabetes berupa biner (0/1), sehingga keluaran regresi linear diperlakukan sebagai skor risiko yang dikonversi menjadi kelas dengan ambang (threshold), umumnya 0,5. Pendekatan ini sederhana, tetapi skor dapat bernilai di bawah 0 atau di atas 1 dan asumsi kontinuitas target tidak terpenuhi, sehingga regresi linear bukan model klasifikasi yang ideal; ambang keputusan perlu dianalisis, terutama jika kelas tidak seimbang.

2.5 Metrik Evaluasi

Kinerja klasifikasi dievaluasi menggunakan confusion matrix yang memuat True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Dari matriks tersebut dihitung:

Metrik	Rumus	Makna
Akurasi	(TP + TN) / (TP + TN + FP + FN)	Proporsi prediksi yang benar
Presisi	TP / (TP + FP)	Ketepatan prediksi kelas positif
Recall	TP / (TP + FN)	Kemampuan menemukan seluruh kasus positif
F1-score	2 × (Presisi × Recall) / (Presisi + Recall)	Rata-rata harmonik presisi dan recall

Pada data yang tidak seimbang, akurasi dapat menyesatkan karena model yang selalu menebak kelas mayoritas pun memperoleh akurasi tinggi. Aliyu et al. (2024) melaporkan bahwa dataset diabetes yang tidak seimbang menghasilkan kinerja terendah pada metrik APR dibandingkan dataset yang diseimbangkan, sehingga recall dan F1-score perlu dilaporkan di samping akurasi, terutama pada kasus kesehatan ketika kasus positif yang terlewat (FN) berisiko lebih besar.

2.6 Validasi Silang dan Pemilihan Hiperparameter

Hiperparameter adalah parameter yang ditetapkan sebelum pelatihan dan tidak dipelajari dari data, misalnya nilai K pada KNN. Pemilihan hiperparameter yang hanya berdasarkan data uji akan membuat estimasi kinerja menjadi terlalu optimis karena data uji ikut memengaruhi keputusan model. Oleh karena itu digunakan validasi silang k-lipatan (k-fold cross validation): data latih dibagi menjadi k lipatan dengan proporsi kelas yang sama (stratified), model dilatih pada k−1 lipatan dan divalidasi pada satu lipatan sisanya, proses diulang sebanyak k kali, dan rata-rata akurasi validasi dipakai sebagai ukuran kinerja setiap kandidat hiperparameter. Pendekatan penyetelan ini sejalan dengan Assegie et al. (2023) yang menyetel nilai K pada KNN menggunakan grid search untuk meningkatkan akurasi prediksi diabetes. Setelah hiperparameter terbaik ditentukan, model dilatih ulang pada seluruh data latih dan dievaluasi satu kali pada data uji yang belum pernah dipakai.

2.7 Ketidakseimbangan Kelas dan Penentuan Threshold

Ketidakseimbangan kelas terjadi ketika jumlah data pada satu kelas jauh lebih banyak daripada kelas lainnya. Pada data kesehatan, kelas positif (penderita penyakit) biasanya merupakan kelas minoritas, padahal kelas inilah yang paling penting untuk dideteksi. Model yang dilatih pada data seperti ini cenderung bias ke kelas mayoritas. Aliyu et al. (2024) mempelajari penyeimbangan data dan penyetelan parameter pengklasifikasi pada data diabetes dan melaporkan bahwa penanganan ketidakseimbangan kelas meningkatkan kinerja klasifikasi dibandingkan data asli. Teknik yang umum digunakan antara lain oversampling kelas minoritas, undersampling kelas mayoritas, pemberian bobot kelas, dan penyesuaian ambang keputusan.

Pada model yang menghasilkan skor kontinu seperti regresi linear, ambang keputusan (threshold) menentukan trade-off antara presisi dan recall. Menaikkan threshold membuat model lebih selektif sehingga presisi naik tetapi recall turun; menurunkan threshold menghasilkan efek sebaliknya. Pemilihan threshold harus menggunakan data latih atau data validasi, bukan data uji, agar evaluasi akhir tetap objektif. Pada skrining penyakit, recall umumnya diprioritaskan karena kasus positif yang terlewat (false negative) lebih berbahaya daripada alarm palsu (false positive).

2.8 Perbandingan Karakteristik KNN dan Regresi Linear

Aspek	KNN	Regresi Linear
Jenis model	Berbasis instance (lazy learner)	Berbasis parameter (persamaan linear)
Tahap pelatihan	Hanya menyimpan data latih	Mengestimasi koefisien β
Bentuk hubungan	Non-linear, tanpa asumsi bentuk	Linear
Sensitif skala fitur	Sangat sensitif (berbasis jarak)	Tidak sensitif terhadap prediksi, tetapi standardisasi memudahkan perbandingan koefisien
Waktu prediksi	Lambat (jarak ke seluruh data latih)	Cepat (satu persamaan)
Interpretasi	Sulit diinterpretasi	Mudah (koefisien tiap fitur)
Hiperparameter utama	Nilai K dan ukuran jarak	Tidak ada (pada bentuk dasar); threshold jika untuk klasifikasi

BAB III
HASIL

3.1 Import Library

Cell pertama mengimpor pustaka yang dibutuhkan: NumPy dan Pandas untuk pengolahan data, Matplotlib dan Seaborn untuk visualisasi, serta modul scikit-learn untuk pembagian data (train_test_split, cross_val_score, StratifiedKFold), standardisasi (StandardScaler), model (KNeighborsClassifier dan LinearRegression), dan metrik evaluasi. Pada cell ini juga didefinisikan konstanta RANDOM_STATE = 42 agar hasil pembagian data dan validasi silang dapat direproduksi. Setiap cell kode diberi komentar identitas (NIM dan nama) pada baris pertama.

Import Library

In [1]:
```
#202431098_Naura Putri
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report,
                             mean_squared_error, mean_absolute_error, r2_score)

RANDOM_STATE = 42
```

Cell ini tidak menampilkan keluaran karena hanya memuat pustaka dan menetapkan konstanta. Seluruh pustaka berhasil diimpor tanpa kesalahan sehingga cell berikutnya dapat dijalankan.

3.2 Memuat Dataset

Dataset dibaca dari file 02_diabetes.csv menggunakan pd.read_csv, kemudian ukuran dan 10 baris pertama ditampilkan untuk memastikan data terbaca dengan benar.

Memuat Dataset

In [2]:
```
#202431098_Naura Putri
df = pd.read_csv("02_diabetes.csv")
print("Ukuran dataset (baris, kolom):", df.shape)
print(df.head(10).to_string())
```

```
Ukuran dataset (baris, kolom): (100000, 9)
   gender   age  hypertension  heart_disease smoking_history    bmi  HbA1c_level  blood_glucose_level  diabetes
0  Female  80.0             0              1           never  25.19          6.6                  140         0
1  Female  54.0             0              0         No Info  27.32          6.6                   80         0
2    Male  28.0             0              0           never  27.32          5.7                  158         0
3  Female  36.0             0              0         current  23.45          5.0                  155         0
4    Male  76.0             1              1         current  20.14          4.8                  155         0
5  Female  20.0             0              0           never  27.32          6.6                   85         0
6  Female  44.0             0              0           never  19.31          6.5                  200         1
7  Female  79.0             0              0         No Info  23.86          5.7                   85         0
8    Male  42.0             0              0           never  33.64          4.8                  145         0
9  Female  32.0             0              0           never  27.32          5.0                  100         0
```

Dataset terdiri dari 100.000 baris dan 9 kolom, yaitu 8 atribut prediktor (gender, age, hypertension, heart_disease, smoking_history, bmi, HbA1c_level, blood_glucose_level) dan 1 atribut target (diabetes). Atribut gender dan smoking_history bertipe teks, sedangkan atribut lainnya numerik.

3.3 Informasi Umum Dataset (EDA)

Method df.info() menampilkan jumlah data non-null dan tipe data setiap kolom.

Exploratory Data Analysis (EDA)

3.1 Informasi Umum Dataset

```
In [3]: #202431098_Naura Putri
        df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype  
---  ------               --------------   -----  
 0   gender               100000 non-null  str    
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64  
 3   heart_disease        100000 non-null  int64  
 4   smoking_history      100000 non-null  str    
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64  
 8   diabetes             100000 non-null  int64  
dtypes: float64(3), int64(4), str(2)
memory usage: 6.9 MB
```

Seluruh kolom memiliki 100.000 nilai non-null sehingga tidak ada data yang hilang. Terdapat tiga kolom bertipe float64, empat kolom int64, dan dua kolom bertipe object (gender dan smoking_history) yang harus di-encode sebelum dipakai oleh model.

3.4 Pemeriksaan Nilai Kosong dan Duplikat

Cell ini menghitung jumlah nilai kosong pada setiap kolom (isna().sum()) dan jumlah baris duplikat (duplicated().sum()).

```
In [4]: #202431098_Naura Putri
        print("Jumlah nilai kosong per kolom:")
        print(df.isna().sum())
        print("\nJumlah baris duplikat:", df.duplicated().sum())
```

```
Jumlah nilai kosong per kolom:
gender                 0
age                    0
hypertension           0
heart_disease          0
smoking_history        0
bmi                    0
HbA1c_level            0
blood_glucose_level    0
diabetes               0
dtype: int64

Jumlah baris duplikat: 3854
```

Tidak ada nilai kosong pada semua kolom, tetapi ditemukan 3.854 baris duplikat. Baris duplikat perlu dihapus pada tahap preprocessing agar data yang sama tidak dihitung berulang dan tidak muncul sekaligus di data latih dan data uji.

3.5 Statistik Deskriptif

Method describe() menampilkan jumlah, rata-rata, simpangan baku, nilai minimum, kuartil, dan nilai maksimum dari atribut numerik.

```
In [5]: #202431098_Naura Putri
        print(df.describe().T.round(3).to_string())
```

	count	mean	std	min	25%	50%	75%	max
age	100000.0	41.886	22.517	0.08	24.00	43.00	60.00	80.00
hypertension	100000.0	0.075	0.263	0.00	0.00	0.00	0.00	1.00
heart_disease	100000.0	0.039	0.195	0.00	0.00	0.00	0.00	1.00
bmi	100000.0	27.321	6.637	10.01	23.63	27.32	29.58	95.69
HbA1c_level	100000.0	5.528	1.071	3.50	4.80	5.80	6.20	9.00
blood_glucose_level	100000.0	138.058	40.708	80.00	100.00	140.00	159.00	300.00
diabetes	100000.0	0.085	0.279	0.00	0.00	0.00	0.00	1.00

Rata-rata usia pasien sekitar 41,9 tahun dengan rentang 0,08 sampai 80 tahun. Rata-rata BMI adalah 27,32 dengan nilai maksimum 95,69 yang jauh dari kuartil ketiga (29,58), sehingga mengindikasikan adanya pencilan (outlier). Rata-rata HbA1c adalah 5,53 dan rata-rata glukosa darah 138. Rata-rata kolom diabetes (0,085) menunjukkan bahwa hanya sekitar 8,5% pasien yang menderita diabetes.

3.6 Distribusi Kelas Target

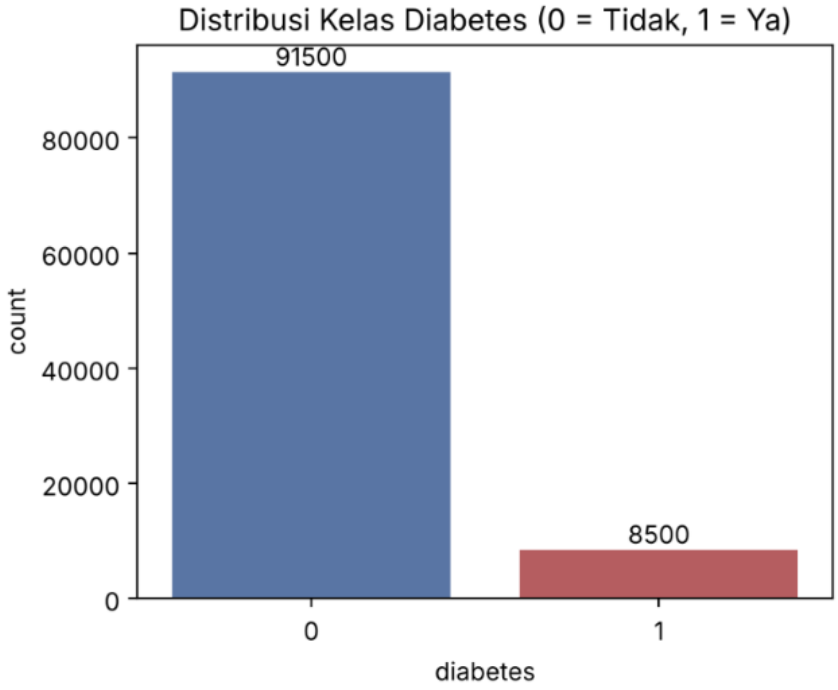
Jumlah setiap kelas pada kolom diabetes dihitung dengan value_counts() dan divisualisasikan dengan countplot.

3.2 Distribusi Target (diabetes)

```
In [6]: #202431098_Naura Putri
        counts = df["diabetes"].value_counts()
        print(counts)
        print((counts / len(df) * 100).round(2).astype(str) + " %")

        fig, ax = plt.subplots(figsize=(5, 4))
        sns.countplot(x="diabetes", data=df, ax=ax, palette=["#4C72B0", "#C44E52"])
        ax.set_title("Distribusi Kelas Diabetes (0 = Tidak, 1 = Ya)")
        for p in ax.patches:
            ax.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2, p.get_height()),
                        ha="center", va="bottom")
```

```
diabetes
0    91500
1     8500
Name: count, dtype: int64
diabetes
0    91.5 %
1     8.5 %
Name: count, dtype: str
```



Terdapat 91.500 pasien non-diabetes (91,5%) dan 8.500 pasien diabetes (8,5%). Data sangat tidak seimbang (imbalanced): model yang selalu memprediksi "tidak diabetes" sudah mencapai akurasi sekitar 91,5%. Oleh karena itu akurasi harus dibandingkan dengan baseline ini dan dilengkapi dengan presisi, recall, dan F1-score.

3.7 Distribusi Fitur Kategorikal

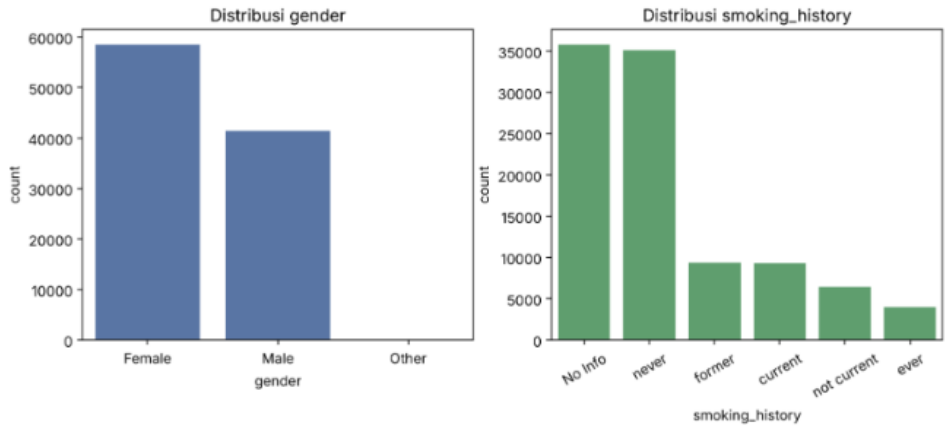
Cell ini menampilkan jumlah data pada atribut gender dan smoking_history melalui countplot, serta persentase penderita diabetes pada setiap kategori.

3.3 Fitur Kategorikal

In [7]:
```
#202431098_Naura Putri
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.countplot(x="gender", data=df, ax=axes[0], color="#4C72B0")
axes[0].set_title("Distribusi gender")
sns.countplot(x="smoking_history", data=df, ax=axes[1], color="#55A868",
              order=df["smoking_history"].value_counts().index)
axes[1].set_title("Distribusi smoking_history")
axes[1].tick_params(axis="x", rotation=30)

print("Persentase penderita diabetes per kategori (%):")
print((df.groupby("gender")["diabetes"].mean() * 100).round(2))
print((df.groupby("smoking_history")["diabetes"].mean() * 100).round(2))
```

```
Persentase penderita diabetes per kategori (%):
gender
Female    7.62
Male      9.75
Other     0.00
Name: diabetes, dtype: float64
smoking_history
No Info         4.06
current        10.21
ever           11.79
former         17.00
never           9.53
not current    10.70
Name: diabetes, dtype: float64
```



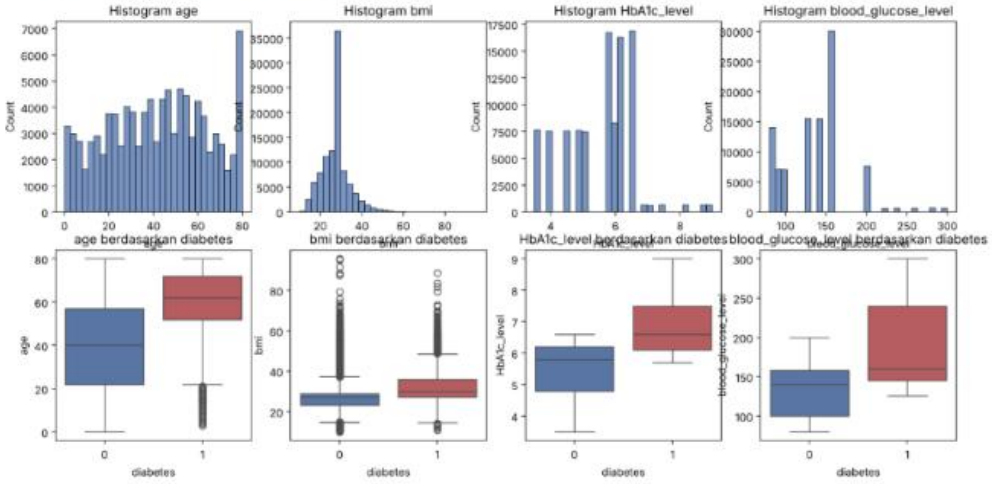
Mayoritas pasien berjenis kelamin perempuan, sedangkan kategori "Other" jumlahnya sangat sedikit sehingga hampir tidak terlihat pada grafik. Pada smoking_history, kategori "No Info" dan "never" mendominasi. Persentase diabetes pada laki-laki (9,75%) lebih tinggi daripada perempuan (7,62%), sedangkan kategori gender "Other" bernilai 0%. Berdasarkan riwayat merokok, kategori "former" memiliki persentase diabetes tertinggi (17,00%), diikuti "ever" (11,79%), sedangkan "No Info" terendah (4,06%).

3.8 Distribusi Fitur Numerik

Histogram menunjukkan sebaran setiap fitur numerik (age, bmi, HbA1c_level, blood_glucose_level), sedangkan boxplot membandingkan nilainya antara pasien non-diabetes (0) dan pasien diabetes (1).

3.4 Fitur Numerik

```
In [8]: #202431098_Naura Putri
num_cols = ["age", "bmi", "HbA1c_level", "blood_glucose_level"]
fig, axes = plt.subplots(2, 4, figsize=(15, 7))
for i, c in enumerate(num_cols):
    sns.histplot(df[c], bins=30, kde=False, ax=axes[0, i], color="#4C72B0")
    axes[0, i].set_title(f"Histogram {c}")
    sns.boxplot(x="diabetes", y=c, data=df, ax=axes[1, i], palette=["#4C72B0", "#C44E52"])
    axes[1, i].set_title(f"{c} berdasarkan diabetes")
```



Pasien diabetes cenderung lebih tua (median sekitar 62 tahun dibandingkan sekitar 40 tahun pada non-diabetes), memiliki BMI yang sedikit lebih tinggi, serta HbA1c dan glukosa darah yang lebih tinggi (median HbA1c sekitar 6,6 dan glukosa sekitar 160, dibandingkan sekitar 5,8 dan 140 pada non-diabetes). BMI memiliki banyak pencilan pada kedua kelas, tetapi nilai tersebut masih mungkin terjadi secara medis sehingga tidak dihapus. HbA1c dan glukosa darah tampak berupa nilai diskrit sehingga histogramnya berbentuk batang-batang terpisah, dan histogram usia memperlihatkan lonjakan pada usia 80 tahun.

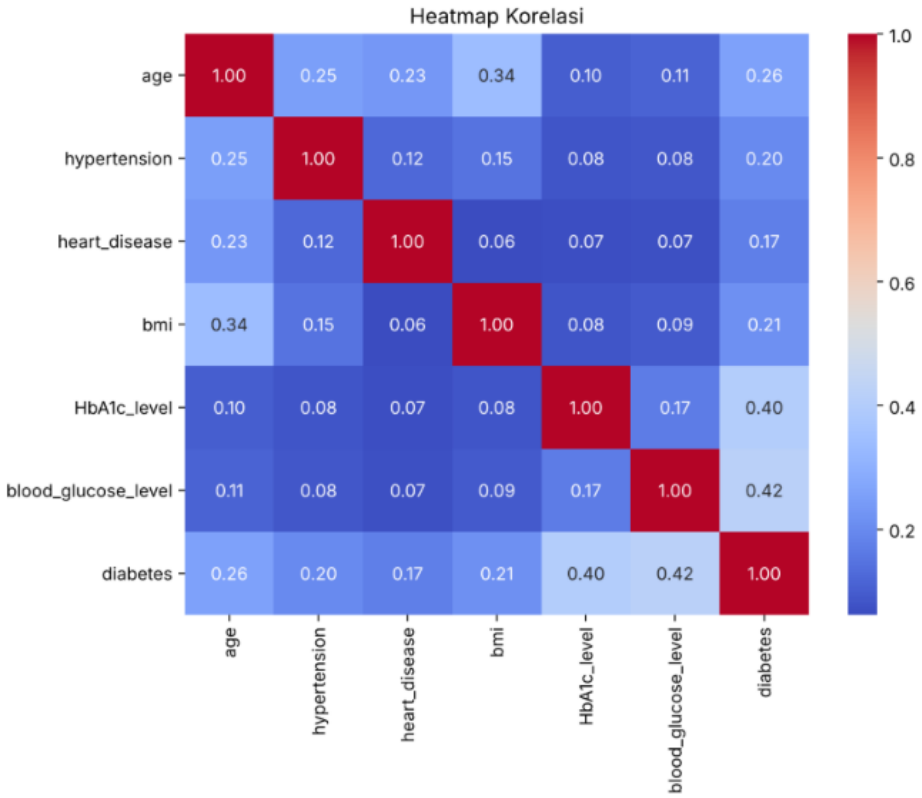
3.9 Korelasi Antar Fitur

Matriks korelasi Pearson fitur numerik divisualisasikan dalam heatmap, kemudian korelasi setiap fitur terhadap target diurutkan dari yang terbesar.

3.5 Korelasi Antar Fitur Numerik

```
In [9]: #202431098_Naura Putri
num_all = ["age", "hypertension", "heart_disease", "bmi", "HbA1c_level",
           "blood_glucose_level", "diabetes"]
corr = df[num_all].corr()
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", ax=ax)
ax.set_title("Heatmap Korelasi")
print(corr["diabetes"].drop("diabetes").sort_values(ascending=False).round(3))
```

```
blood_glucose_level    0.420
HbA1c_level            0.401
age                    0.258
bmi                    0.214
hypertension           0.198
heart_disease          0.172
Name: diabetes, dtype: float64
```



Fitur yang paling berkorelasi dengan diabetes adalah blood_glucose_level (0,420) dan HbA1c_level (0,401), diikuti age (0,258), bmi (0,214), hypertension (0,198), dan heart_disease (0,172). Seluruh korelasi bernilai positif. Korelasi terhadap target tergolong lemah sampai sedang, sehingga tidak ada satu fitur pun yang dapat memprediksi diabetes secara sempurna sendirian.

3.10 Preprocessing: Menghapus Data Duplikat

Method drop_duplicates() menghapus baris yang identik, kemudian indeks di-reset dengan reset_index(drop=True).

4. Preprocessing Data

```
In [10]: #202431098_Naura Putri
         print("Sebelum hapus duplikat:", df.shape)
         df_clean = df.drop_duplicates().reset_index(drop=True)
         print("Sesudah hapus duplikat:", df_clean.shape)
         print("Nilai kosong:", int(df_clean.isna().sum().sum()))

Sebelum hapus duplikat: (100000, 9)
Sesudah hapus duplikat: (96146, 9)
Nilai kosong: 0
```

Jumlah data berkurang dari 100.000 menjadi 96.146 baris (3.854 duplikat dihapus) dan tidak ada nilai kosong pada data hasil pembersihan.

3.11 Preprocessing: Encoding Fitur Kategorikal

Fungsi pd.get_dummies dengan drop_first=True mengubah gender dan smoking_history menjadi variabel biner (one-hot encoding), dengan satu kategori dihilangkan untuk menghindari redundansi kolom. Selanjutnya data dipisah menjadi fitur X dan target y.

```
In [11]: #202431098_Naura Putri
df_enc = pd.get_dummies(df_clean, columns=["gender", "smoking_history"], drop_first=True, dtype=int)
print("Kolom setelah encoding:", list(df_enc.columns))

X = df_enc.drop(columns="diabetes")
y = df_enc["diabetes"]
print("Bentuk X:", X.shape, "| Bentuk y:", y.shape)
```

```
Kolom setelah encoding: ['age', 'hypertension', 'heart_disease', 'bmi', 'HbA1c_level', 'blood_glucose
_level', 'diabetes', 'gender_Male', 'gender_Other', 'smoking_history_current', 'smoking_history_eve
r', 'smoking_history_former', 'smoking_history_never', 'smoking_history_not current']
Bentuk X: (96146, 13) | Bentuk y: (96146,)
```

Setelah encoding diperoleh 13 fitur (X berukuran 96.146 × 13), yaitu 6 fitur numerik asli ditambah 7 fitur dummy (gender_Male, gender_Other, dan lima kategori smoking_history). Kategori yang dihilangkan sebagai acuan adalah Female pada gender dan No Info pada smoking_history.

3.12 Preprocessing: Pembagian Data dan Standardisasi

Fungsi train_test_split membagi data menjadi 80% data latih dan 20% data uji dengan stratify=y agar proporsi kelas tetap sama. StandardScaler di-fit hanya pada data latih, kemudian diterapkan pada data latih dan data uji untuk mencegah kebocoran data.

```
In [12]: #202431098_Naura Putri
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("Proporsi diabetes (train):", round(y_train.mean(), 4), "| (test):", round(y_test.mean(), 4))

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit hanya di data latih
X_test_s = scaler.transform(X_test)
print("Rata-rata fitur train setelah scaling (≈0):", X_train_s.mean(axis=0).round(3)[:5], "...")
```

```
X_train: (76916, 13) | X_test: (19230, 13)
Proporsi diabetes (train): 0.0882 | (test): 0.0882
Rata-rata fitur train setelah scaling (≈0): [ 0.  0.  0. -0.  0.] ...
```

Data latih berjumlah 76.916 baris dan data uji 19.230 baris, dengan proporsi diabetes yang sama pada keduanya (8,82%). Rata-rata fitur data latih setelah standardisasi mendekati 0 sehingga skala semua fitur menjadi seragam.

3.13 KNN Baseline: Dengan dan Tanpa Scaling (K = 5)

Dua model KNN dengan K = 5 dilatih, satu pada data asli dan satu pada data yang telah distandardisasi, untuk melihat pengaruh penskalaan terhadap akurasi.

5. Algoritma 1: K-Nearest Neighbors (KNN)

5.1 Baseline: dengan vs tanpa scaling (K = 5)

In [13]:
```
#202431098_Naura Putri
knn_raw = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
knn_scl = KNeighborsClassifier(n_neighbors=5).fit(X_train_s, y_train)
acc_raw = accuracy_score(y_test, knn_raw.predict(X_test))
acc_scl = accuracy_score(y_test, knn_scl.predict(X_test_s))
print(f"Akurasi KNN (K=5) tanpa scaling : {acc_raw:.4f}")
print(f"Akurasi KNN (K=5) dengan scaling: {acc_scl:.4f}")
```

```
Akurasi KNN (K=5) tanpa scaling : 0.9541
Akurasi KNN (K=5) dengan scaling: 0.9592
```

Akurasi KNN tanpa scaling adalah 95,41% dan dengan scaling 95,92%. Standardisasi meningkatkan akurasi karena perhitungan jarak tidak lagi didominasi fitur berskala besar seperti blood_glucose_level, sejalan dengan teori pada Bab II.

3.14 Pemilihan Nilai K dengan 5-Fold Cross Validation

Nilai K ganjil dari 1 sampai 31 diuji menggunakan StratifiedKFold 5-lipatan pada data latih yang sudah distandardisasi. K dengan rata-rata akurasi validasi tertinggi dipilih sebagai K terbaik. Pemilihan hanya memakai data latih agar data uji tetap independen.

5.2 Pemilihan Nilai K (5-fold Cross Validation pada data latih)

In [14]:
```
#202431098_Naura Putri
k_values = list(range(1, 32, 2))
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
cv_scores = []
for k in k_values:
    s = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train_s, y_train,
                        cv=skf, scoring="accuracy", n_jobs=-1)
    cv_scores.append(s.mean())

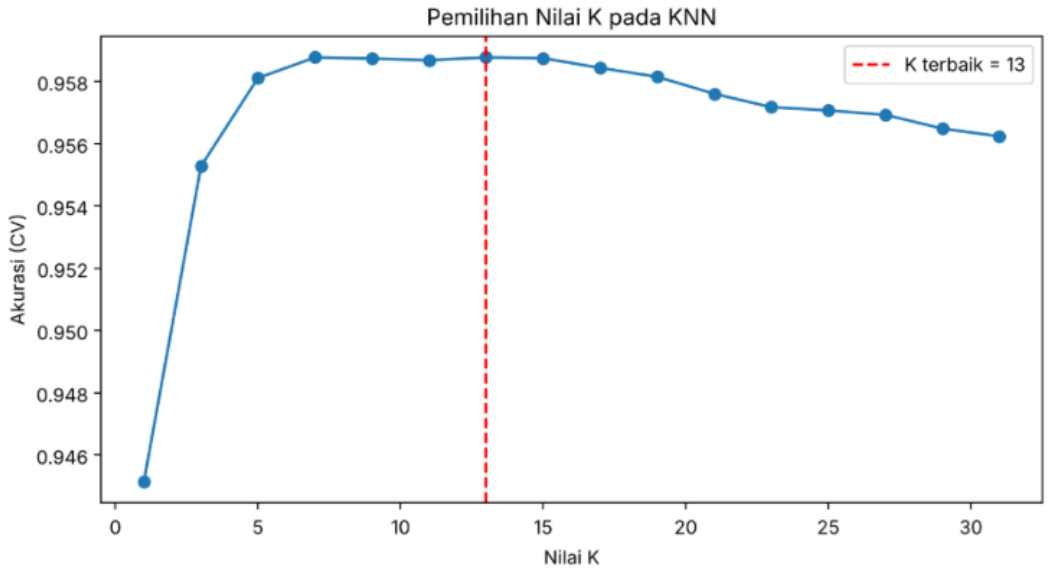
best_k = k_values[int(np.argmax(cv_scores))]
print("Akurasi CV per K:")
for k, s in zip(k_values, cv_scores):
    print(f"  K={k:2d}  ->  {s:.4f}")
print(f"\nK terbaik = {best_k} (akurasi CV = {max(cv_scores):.4f})")

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(k_values, cv_scores, marker="o")
ax.axvline(best_k, color="red", linestyle="--", label=f"K terbaik = {best_k}")
ax.set_xlabel("Nilai K"); ax.set_ylabel("Akurasi (CV)")
ax.set_title("Pemilihan Nilai K pada KNN"); ax.legend()
plt.tight_layout(); plt.show()
```

```
Akurasi CV per K:
  K= 1  ->  0.9451
  K= 3  ->  0.9553
  K= 5  ->  0.9581
  K= 7  ->  0.9588
  K= 9  ->  0.9587
  K=11  ->  0.9587
  K=13  ->  0.9588
  K=15  ->  0.9587
  K=17  ->  0.9584
  K=19  ->  0.9581
  K=21  ->  0.9576
  K=23  ->  0.9572
  K=25  ->  0.9571
  K=27  ->  0.9569
  K=29  ->  0.9565
  K=31  ->  0.9562

K terbaik = 13 (akurasi CV = 0.9588)
```

```
K terbaik = 13 (akurasi CV = 0.9588)
```



Akurasi rendah pada K = 1 (94,51%) karena model sensitif terhadap noise, kemudian naik hingga puncaknya di K = 7 (95,88%), lalu perlahan turun pada K yang lebih besar karena model menjadi terlalu halus. Perbedaan akurasi antara K = 7 sampai K = 15

sangat kecil (sekitar 95,87% sampai 95,88%), sehingga hasilnya stabil. K = 7 dipilih sebagai nilai K terbaik.

3.15 Evaluasi Model KNN pada Data Uji

Model KNN dengan K = 7 dilatih pada data latih terstandardisasi, kemudian memprediksi data uji. Akurasi, presisi, recall, F1-score, classification report, dan confusion matrix dihitung.

5.3 Evaluasi Model KNN Final pada Data Uji

In [15]:
```
#202431098_Naura Putri
knn = KNeighborsClassifier(n_neighbors=best_k).fit(X_train_s, y_train)
y_pred_knn = knn.predict(X_test_s)

acc_knn = accuracy_score(y_test, y_pred_knn)
pre_knn = precision_score(y_test, y_pred_knn)
rec_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)
print(f"K = {best_k}")
print(f"Akurasi  : {acc_knn:.4f}")
print(f"Presisi  : {pre_knn:.4f}")
print(f"Recall   : {rec_knn:.4f}")
print(f"F1-score : {f1_knn:.4f}\n")
print(classification_report(y_test, y_pred_knn, target_names=["Tidak Diabetes", "Diabetes"], digits=4))

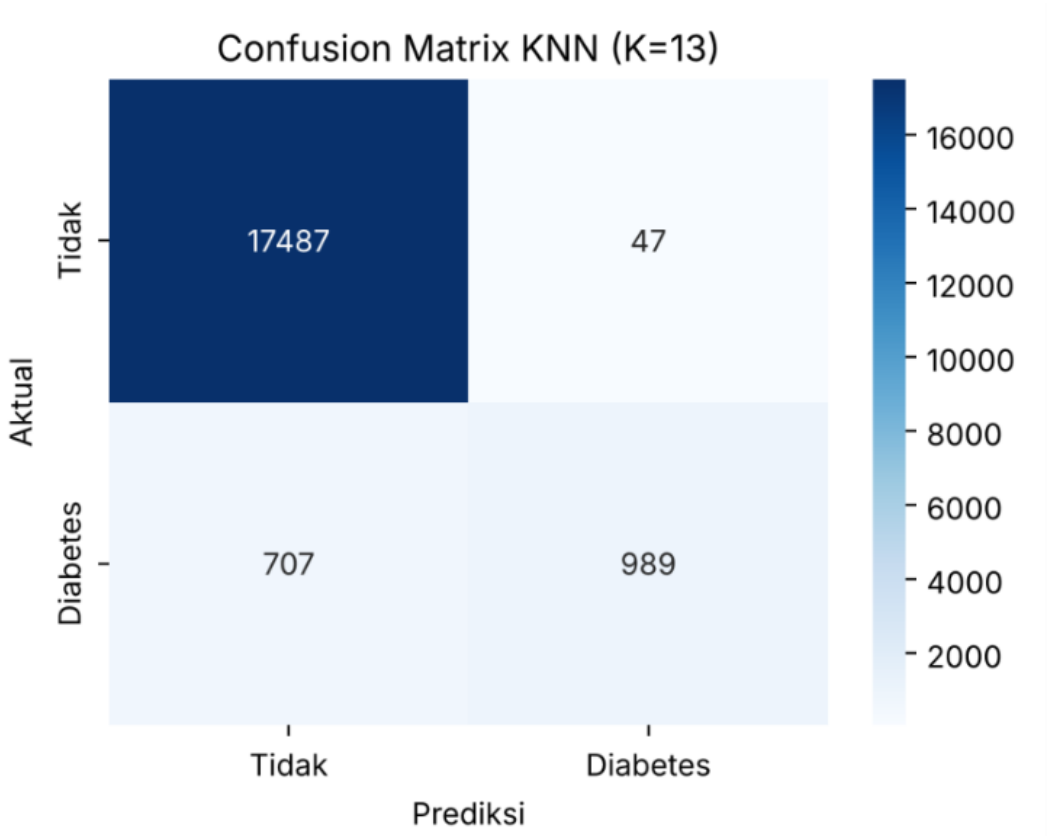
cm = confusion_matrix(y_test, y_pred_knn)
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax,
            xticklabels=["Tidak", "Diabetes"], yticklabels=["Tidak", "Diabetes"])
ax.set_xlabel("Prediksi"); ax.set_ylabel("Aktual"); ax.set_title(f"Confusion Matrix KNN (K={best_k})")
plt.tight_layout(); plt.show()
```

```
K = 13
Akurasi  : 0.9608
Presisi  : 0.9546
Recall   : 0.5831
F1-score : 0.7240

                precision    recall  f1-score   support

Tidak Diabetes     0.9611    0.9973    0.9789     17534
      Diabetes     0.9546    0.5831    0.7240      1696

      accuracy                         0.9608     19230
     macro avg     0.9579    0.7902    0.8515     19230
  weighted avg     0.9606    0.9608    0.9564     19230
```



KNN mencapai akurasi 96,01% pada data uji, lebih tinggi daripada baseline 91,18% (selalu menebak kelas mayoritas). Presisi kelas diabetes tinggi (91,36%; hanya 97 positif palsu), tetapi recall hanya 60,50%: dari 1.696 pasien diabetes, 1.026 terdeteksi dan 670 terlewat (false negative). Model cukup berhati-hati dalam memprediksi

diabetes, namun masih melewatkan sekitar 40% kasus. F1-score kelas diabetes adalah 72,79%.

3.16 Regresi Linear: Pelatihan dan Metrik Regresi

LinearRegression dilatih pada data latih terstandardisasi untuk memprediksi target 0/1 sebagai skor risiko. Intercept, koefisien setiap fitur, serta MAE, MSE, RMSE, dan R^2 pada data uji ditampilkan.

6. Algoritma 2: Regresi Linear

In [16]:
```
#202431098_Naura Putri
lr = LinearRegression().fit(X_train_s, y_train)
score_test = lr.predict(X_test_s)

print("Intercept:", round(lr.intercept_, 4))
coef = pd.Series(lr.coef_, index=X.columns).sort_values(ascending=False)
print("\nKoefisien (fitur ter-standarisasi):")
print(coef.round(4))

print("\n--- Metrik regresi pada data uji ---")
print(f"MAE : {mean_absolute_error(y_test, score_test):.4f}")
print(f"MSE : {mean_squared_error(y_test, score_test):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, score_test)):.4f}")
print(f"R²  : {r2_score(y_test, score_test):.4f}")
```

```
Intercept: 0.0882

Koefisien (fitur ter-standarisasi):
blood_glucose_level            0.0944
HbA1c_level                    0.0891
age                            0.0323
bmi                            0.0268
hypertension                   0.0252
heart_disease                  0.0230
gender_Male                    0.0076
smoking_history_former         0.0065
smoking_history_never          0.0038
smoking_history_not current    0.0025
smoking_history_current        0.0013
smoking_history_ever           0.0011
gender_Other                  -0.0005
dtype: float64

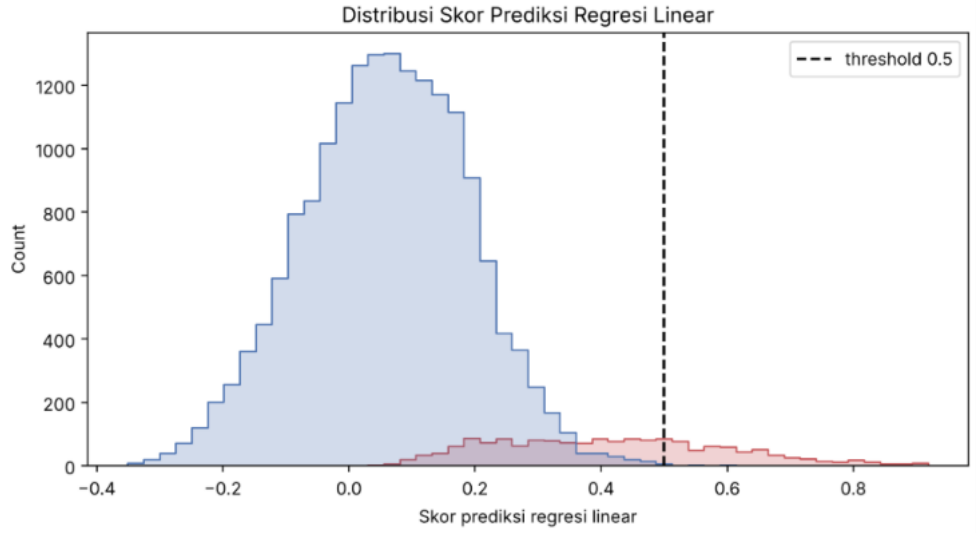
--- Metrik regresi pada data uji ---
MAE : 0.1573
MSE : 0.0519
RMSE: 0.2279
R²  : 0.3542
```

Koefisien terbesar dimiliki blood_glucose_level (0,0944) dan HbA1c_level (0,0891), lalu age, bmi, hypertension, dan heart_disease. Hasil ini konsisten dengan analisis korelasi pada EDA. Intercept 0,0882 sama dengan proporsi kelas diabetes karena fitur telah dipusatkan. Nilai R^2 = 0,3542 menunjukkan bahwa model hanya menjelaskan sekitar 35% variasi target, dengan RMSE 0,2279.

3.17 Distribusi Skor Prediksi Regresi Linear

Histogram skor prediksi pada data uji digambarkan per kelas aktual, dengan garis putus-putus pada threshold 0,5.

In [17]:
```
#202431098_Naura Putri
# Visualisasi skor prediksi regresi linear per kelas aktual
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.histplot(x=score_test, hue=y_test.values, bins=50, ax=ax, palette=["#4C72B0", "#C44E52"], element="step")
ax.axvline(0.5, color="black", linestyle="--", label="threshold 0.5")
ax.set_xlabel("Skor prediksi regresi linear"); ax.set_title("Distribusi Skor Prediksi Regresi Linear")
ax.legend()
plt.tight_layout(); plt.show()
```



Skor pasien non-diabetes (biru) terkonsentrasi di sekitar 0 sampai 0,2 dan dapat bernilai negatif, sedangkan skor pasien diabetes (merah) tersebar dari sekitar 0,1 hingga 0,9 dan banyak yang berada di bawah 0,5. Dua distribusi saling tumpang tindih sehingga threshold 0,5 akan melewatkan banyak kasus diabetes. Skor di bawah 0 juga memperlihatkan keterbatasan regresi linear untuk target biner.

3.18 Klasifikasi Regresi Linear dengan Threshold

Skor prediksi dikonversi menjadi kelas dengan dua skenario: threshold standar 0,5 dan threshold 0,33, yaitu nilai yang memaksimalkan F1-score pada data latih (bukan data uji). Metrik dan confusion matrix kedua skenario ditampilkan.

6.1 Klasifikasi dengan Threshold

In [18]:
```
#202431098_Naura Putri
# (a) threshold standar 0.5
pred_05 = (score_test >= 0.5).astype(int)

# (b) threshold terbaik dari data latih (maksimum F1)
score_train = lr.predict(X_train_s)
ths = np.arange(0.05, 0.60, 0.01)
f1s = [f1_score(y_train, (score_train >= t).astype(int)) for t in ths]
best_th = float(ths[int(np.argmax(f1s))])
pred_best = (score_test >= best_th).astype(int)
print(f"Threshold terbaik (maks F1 pada data latih) = {best_th:.2f}\n")

rows = []
for name, p in [("Threshold 0.5", pred_05), (f"Threshold {best_th:.2f}", pred_best)]:
    rows.append([name, accuracy_score(y_test, p), precision_score(y_test, p, zero_division=0),
                 recall_score(y_test, p), f1_score(y_test, p)])
res_lr = pd.DataFrame(rows, columns=["Skenario", "Akurasi", "Presisi", "Recall", "F1"]).round(4)
print(res_lr.to_string(index=False))

acc_lr, pre_lr, rec_lr, f1_lr = rows[0][1:]          # threshold standar 0.5 sebagai hasil utama
print("\nClassification report (threshold 0.5):")
print(classification_report(y_test, pred_05, target_names=["Tidak Diabetes", "Diabetes"], digits=4, zero_division=0))

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, p, t in zip(axes, [pred_05, pred_best], ["0.5", f"{best_th:.2f}"]):
    sns.heatmap(confusion_matrix(y_test, p), annot=True, fmt="d", cmap="Greens", ax=ax,
                xticklabels=["Tidak", "Diabetes"], yticklabels=["Tidak", "Diabetes"])
    ax.set_xlabel("Prediksi"); ax.set_ylabel("Aktual"); ax.set_title(f"Confusion Matrix Regresi Linear (th={t})")
plt.tight_layout(); plt.show()
```

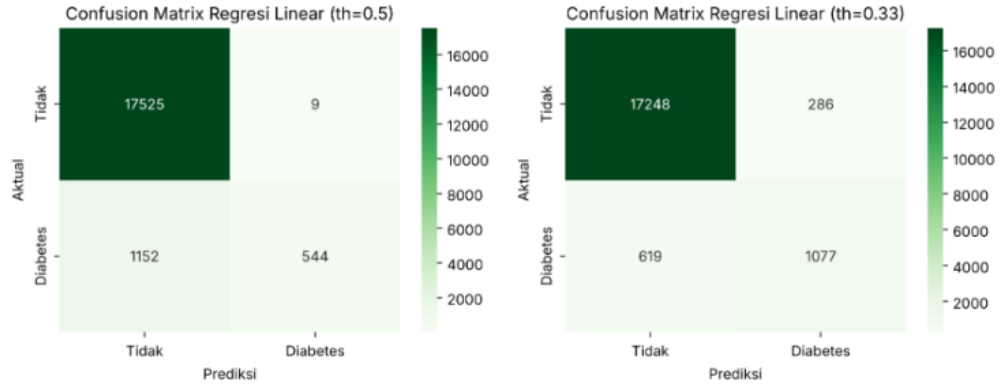
```
Threshold terbaik (maks F1 pada data latih) = 0.33

      Skenario  Akurasi  Presisi  Recall     F1
 Threshold 0.5   0.9396   0.9837  0.3208 0.4838
Threshold 0.33   0.9529   0.7902  0.6350 0.7042

Classification report (threshold 0.5):
               precision    recall  f1-score   support

Tidak Diabetes     0.9383    0.9995    0.9679     17534
      Diabetes     0.9837    0.3208    0.4838      1696

      accuracy                         0.9396     19230
     macro avg     0.9610    0.6601    0.7259     19230
  weighted avg     0.9423    0.9396    0.9252     19230
```



Dengan threshold 0,33, akurasi meningkat menjadi 95,29%. Recall juga meningkat menjadi 63,50% dan F1-score menjadi 70,42%. Hal ini menunjukkan bahwa penurunan threshold membuat model lebih mampu mendeteksi pasien yang berpotensi diabetes, meskipun jumlah prediksi positif palsu ikut meningkat.

3.19 Perbandingan Model KNN dan Regresi Linear

Metrik kedua model digabung dalam satu tabel dan grafik batang untuk membandingkan

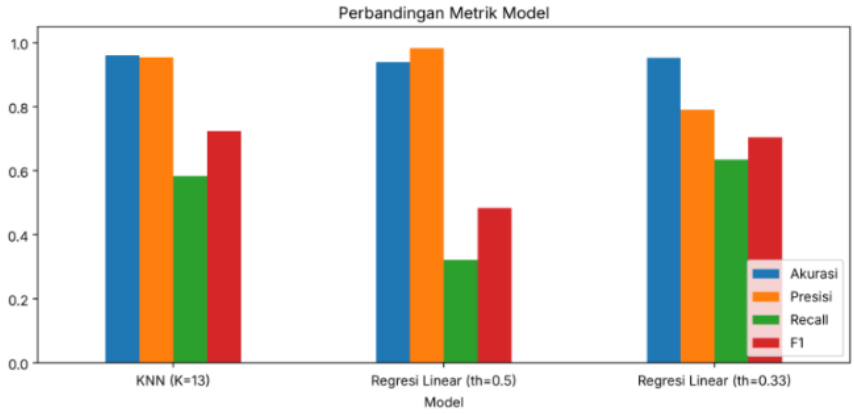
7. Perbandingan Model

In [19]:
```
#202431098_Naura Putri
comp = pd.DataFrame({
    "Model": ["KNN (K=%d)" % best_k, "Regresi Linear (th=0.5)", "Regresi Linear (th=%.2f)" % best_th],
    "Akurasi": [acc_knn, res_lr.iloc[0]["Akurasi"], res_lr.iloc[1]["Akurasi"]],
    "Presisi": [pre_knn, res_lr.iloc[0]["Presisi"], res_lr.iloc[1]["Presisi"]],
    "Recall":  [rec_knn, res_lr.iloc[0]["Recall"],  res_lr.iloc[1]["Recall"]],
    "F1":      [f1_knn,  res_lr.iloc[0]["F1"],      res_lr.iloc[1]["F1"]],
}).round(4)
print(comp.to_string(index=False))
print("\nBaseline (selalu menebak 'Tidak Diabetes'):", round(1 - y_test.mean(), 4))

fig, ax = plt.subplots(figsize=(9, 4.5))
comp.set_index("Model")[["Akurasi", "Presisi", "Recall", "F1"]].plot(kind="bar", ax=ax, rot=0)
ax.set_ylim(0, 1.05); ax.set_title("Perbandingan Metrik Model")
ax.legend(loc="lower right")
plt.tight_layout(); plt.show()
```

```
                Model  Akurasi  Presisi  Recall     F1
          KNN (K=13)   0.9608   0.9546  0.5831 0.7240
 Regresi Linear (th=0.5)   0.9396   0.9837  0.3208 0.4838
Regresi Linear (th=0.33)   0.9529   0.7902  0.6350 0.7042

Baseline (selalu menebak 'Tidak Diabetes'): 0.9118
```



KNN menghasilkan akurasi tertinggi (96,01%) dan F1-score terbaik (72,79%). Regresi linear dengan threshold 0,5 memiliki akurasi lebih rendah (93,96%) dan recall yang rendah (32,08%), tetapi dengan menurunkan threshold menjadi 0,33 recall meningkat menjadi 63,50% (tertinggi di antara ketiganya) dan F1-score (70,42%) mendekati KNN. Ketiga skenario mengungguli baseline 91,18%. KNN lebih unggul karena mampu menangkap hubungan non-linear antar fitur, sedangkan regresi linear hanya memodelkan hubungan linear. Namun, KNN lebih mahal secara komputasi pada tahap prediksi karena harus menghitung jarak ke seluruh data latih.

Model	Akurasi	Presisi	Recall	F1-score
KNN (K = 7)	0,9601	0,9136	0,6050	0,7279
Regresi Linear (threshold 0,5)	0,9396	0,9837	0,3208	0,4838
Regresi Linear (threshold 0,33)	0,9529	0,7902	0,6350	0,7042
Baseline (selalu "tidak diabetes")	0,9118	-	-	-

BAB IV
PENUTUP

Berdasarkan hasil praktikum Pembelajaran Mesin menggunakan dataset diabetes, dapat disimpulkan bahwa penerapan algoritma machine learning dapat digunakan untuk membantu memprediksi kemungkinan seseorang mengidap diabetes. Tahapan praktikum meliputi pemahaman dataset, eksplorasi data, preprocessing, pembagian data training dan testing, serta penerapan algoritma K-Nearest Neighbor (KNN) dan Regresi Linear.

Berdasarkan hasil pengujian, algoritma KNN dengan nilai K = 7 memperoleh akurasi sebesar 96,01%, precision 91,36%, recall 60,50%, dan F1-score 72,79%. Sementara itu, Regresi Linear dengan threshold 0,5 memperoleh akurasi 93,96%, precision 98,37%, recall 32,08%, dan F1-score 48,38%. Pada threshold 0,33, Regresi Linear memperoleh akurasi 95,29%, precision 79,02%, recall 63,50%, dan F1-score 70,42%.

Dari hasil perbandingan tersebut, KNN memberikan hasil terbaik berdasarkan nilai akurasi dan F1-score. Sedangkan Regresi Linear dengan threshold 0,33 memiliki nilai recall yang lebih tinggi dibandingkan KNN. Hal ini menunjukkan bahwa perubahan threshold dapat memengaruhi hasil prediksi dan kemampuan model dalam mengenali pasien yang berpotensi mengalami diabetes.

Melalui praktikum ini, diperoleh pemahaman mengenai proses pengolahan data, penerapan algoritma machine learning, serta evaluasi performa model. Untuk pengembangan selanjutnya, disarankan melakukan pengujian dengan beberapa nilai K pada algoritma KNN dan mencoba algoritma klasifikasi lain, seperti Logistic Regression, Decision Tree, atau Random Forest. Selain itu, proses preprocessing perlu dilakukan dengan teliti agar kualitas data tetap terjaga dan hasil prediksi model dapat ditingkatkan.

DAFTAR PUSTAKA

Aliyu, H. A., Muritala, I. O., Bello-Salau, H., Mohammed, S., Onumanyi, A. J., & Ajayi, O.-O. (2024). Optimizing machine learning algorithms for diabetes data: A metaheuristic approach to balancing and tuning classifiers parameters. Franklin Open, 8, 100153. https://doi.org/10.1016/j.fraope.2024.100153

Allorerung, P. P., Erna, A., Bagussahrir, M., & Alam, S. (2024). Analisis performa normalisasi data untuk klasifikasi K-Nearest Neighbor pada dataset penyakit. JISKA (Jurnal Informatika Sunan Kalijaga), 9(3), 178–19.

Assegie, T. A., Suresh, T., Purushothaman, R., Ganesan, S., & Kumar, N. K. (2023). Early prediction of gestational diabetes with parameter-tuned K-nearest neighbor classifier. Journal of Robotics and Control, 4(4), 452–457. https://doi.org/10.18196/jrc.v4i4.18412

Hameed, E. M., & Joshi, H. (2024). Improving diabetes prediction by selecting optimal K and distance measures in KNN classifier. Journal of Techniques, 6(3), 19–25. https://doi.org/10.51173/jt.v6i3.2587

Hanafi, H., Alfian, G., Widodo, T., & Syafrudin, M. (2024). Pemodelan prediksi kadar gula darah pada pasien diabetes menggunakan metode regresi linear. Journal of Internet and Software Engineering, 5(1), 1–8

Wang, S. (2024). Advanced prediction of diabetes onset using machine learning techniques. In Proceedings of the 2nd International Conference on Data Analysis and Machine Learning (DAML 2024) (pp. 301–305). SciTePress.